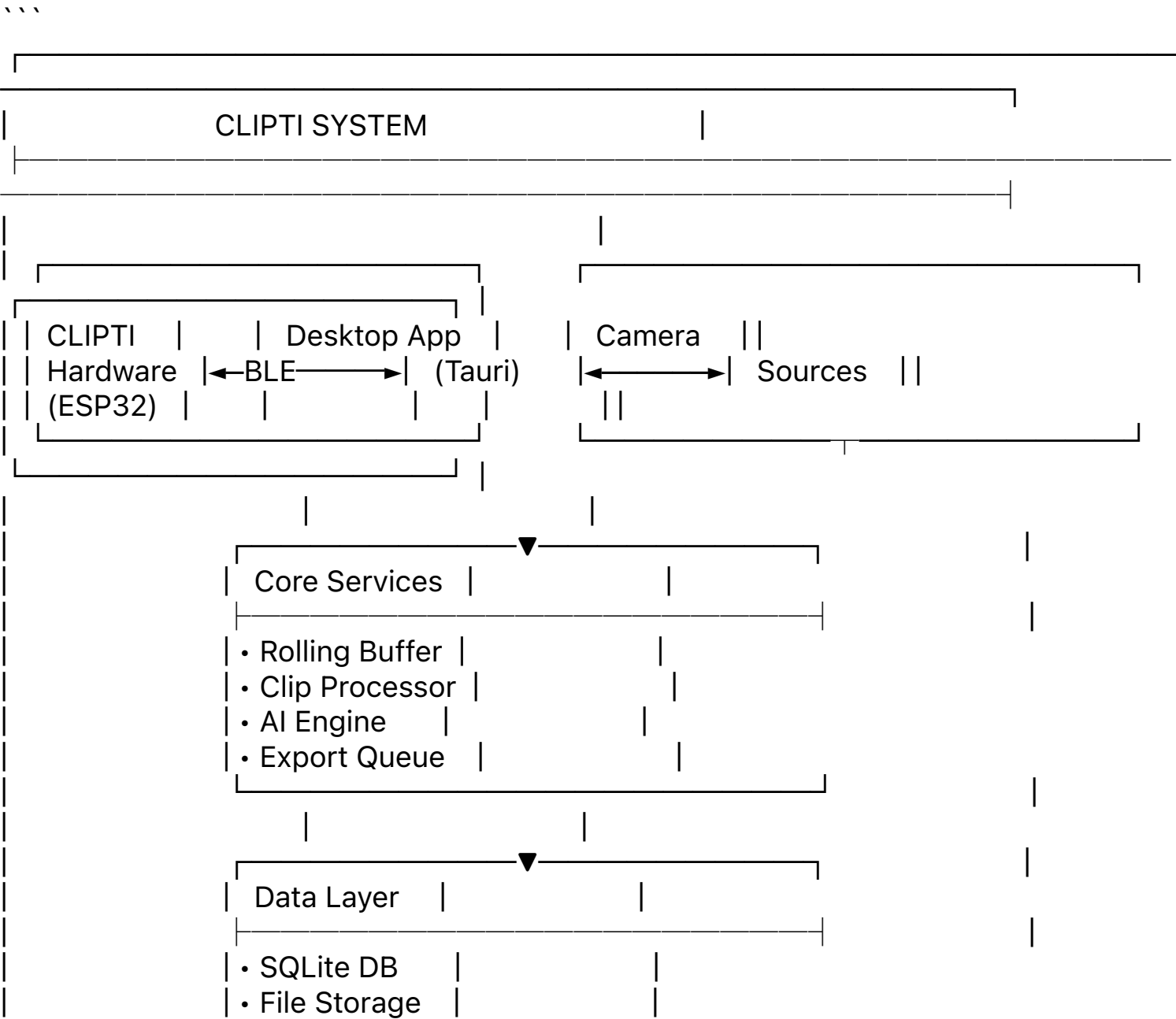


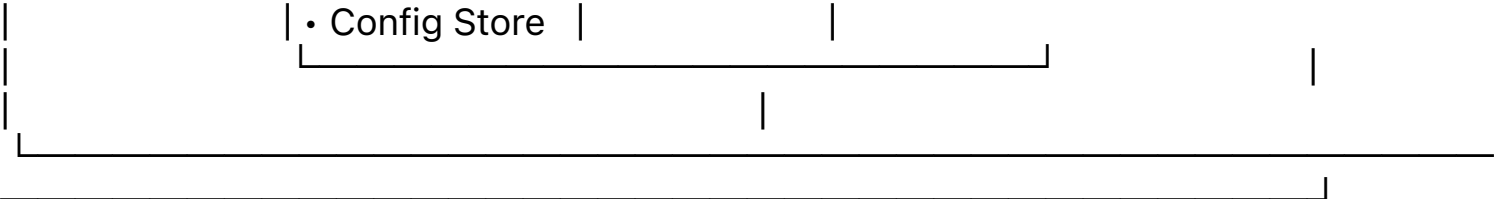
CLIPTI Technical Architecture Specification

Overview

- CLIPTI is an AI-powered NFC clip button system consisting of:
- 1. **Hardware** - ESP32-S3 based NFC button device
 - 2. **Desktop App** - Tauri + React application
 - 3. **Cloud Services** - Optional AI processing (DeepSeek, Whisper)

System Architecture



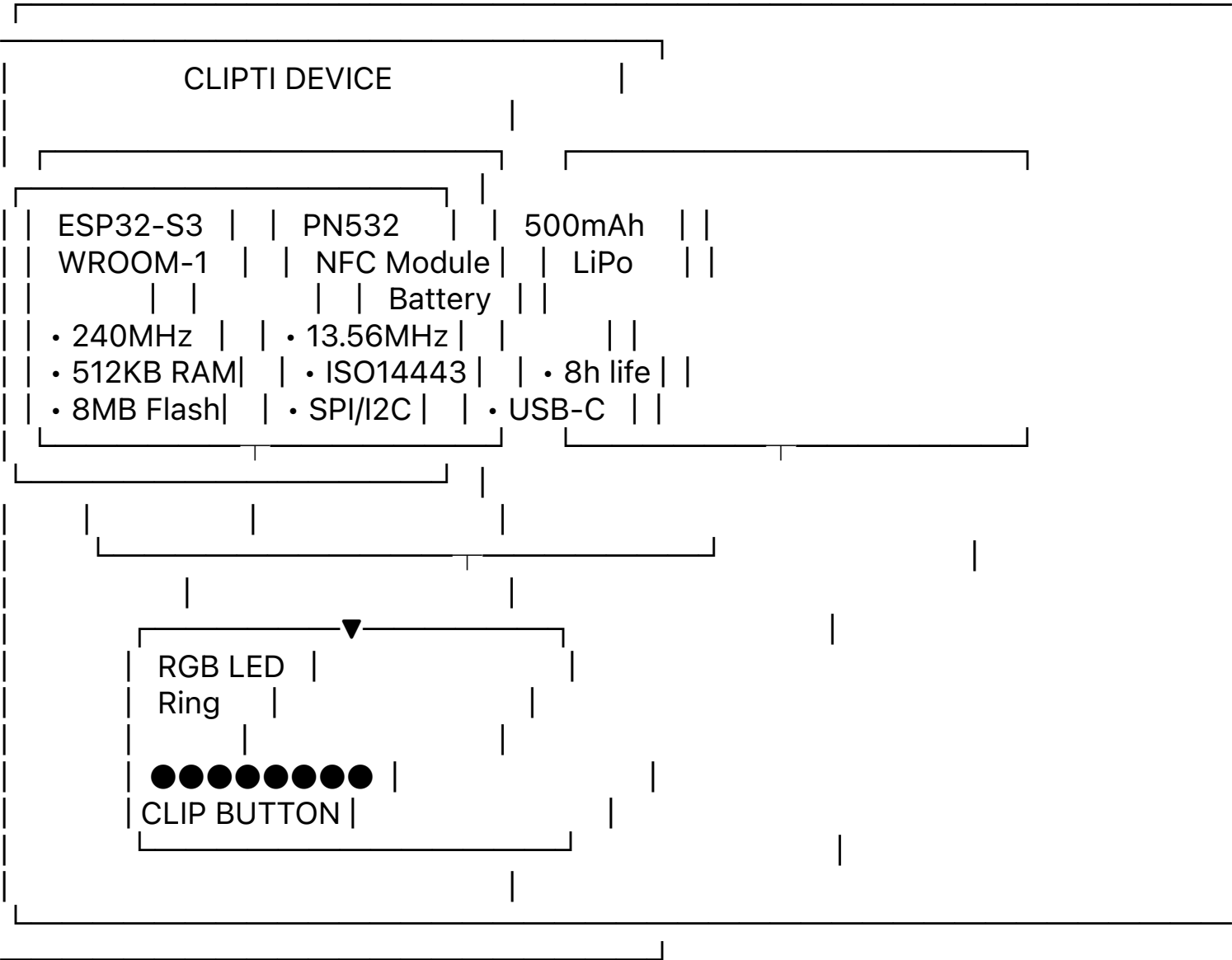


...

Hardware Specification

CLIPTI Device (ESP32-S3)

...

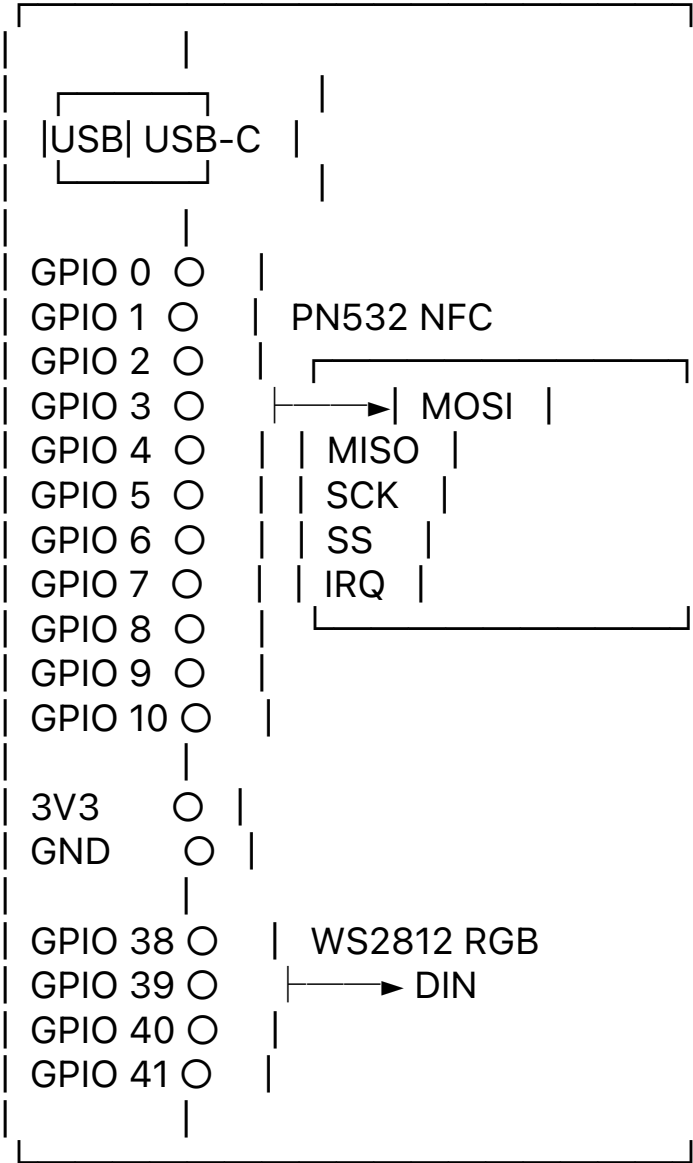


...

Pinout Diagram

...

ESP32-S3-WROOM-1

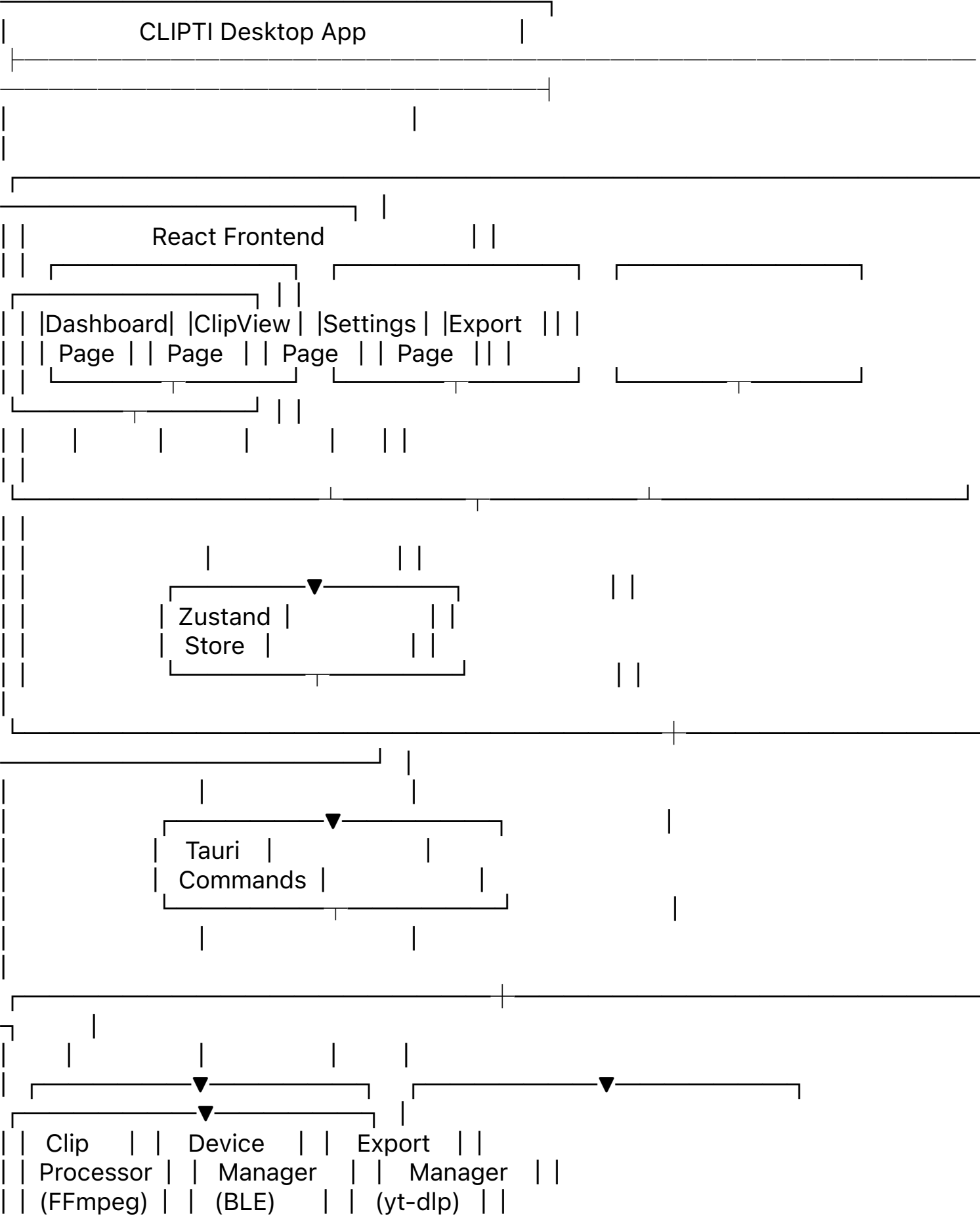


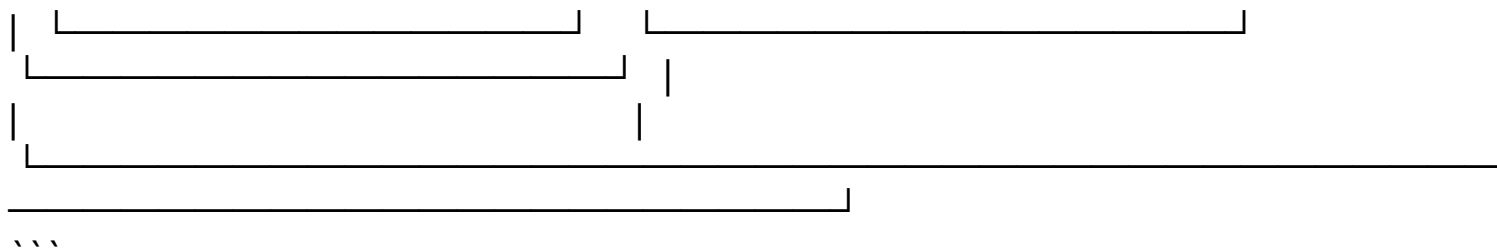
...

Software Architecture

Desktop App Architecture (Tauri)

...





Rust Backend Modules

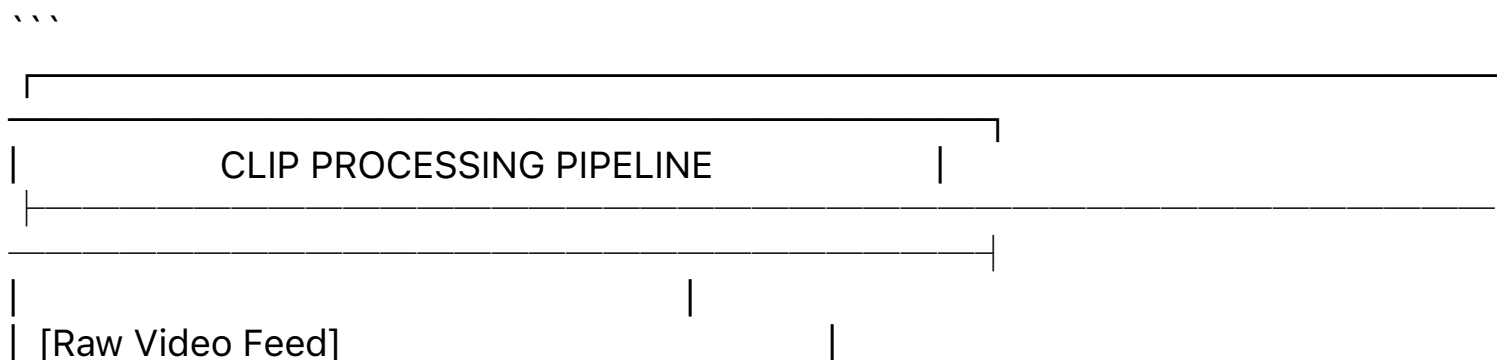
```
```rust
// src-tauri/src/main.rs - Entry point

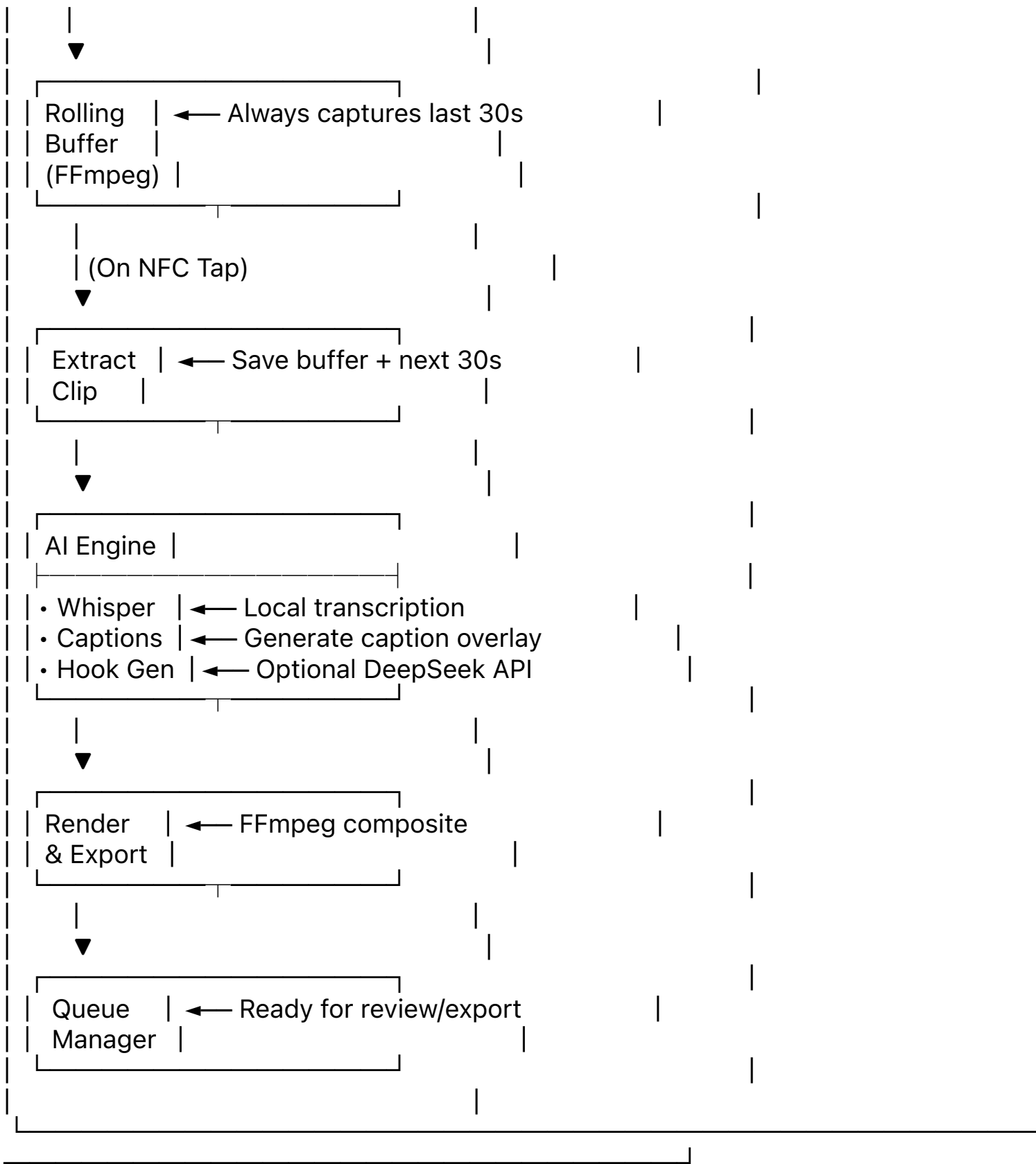
mod clip_processor;
mod device_manager;
mod export_manager;
mod storage;
mod ai_engine;

use clip_processor::{ClipProcessor, ClipMetadata};
use device_manager::DeviceManager;
use export_manager::ExportManager;
use storage::Storage;

// Main application state
pub struct AppState {
 pub clip_processor: ClipProcessor,
 pub device_manager: DeviceManager,
 pub export_manager: ExportManager,
 pub storage: Storage,
}
```
```

Clip Processing Pipeline





...

Data Models

SQLite Schema

```
```sql
-- Clips table
CREATE TABLE clips (
 id TEXT PRIMARY KEY,
 device_id TEXT NOT NULL,
 source_camera TEXT,
 duration_seconds INTEGER,
 file_path TEXT NOT NULL,
 thumbnail_path TEXT,
 status TEXT DEFAULT 'pending', -- pending, processing, ready, exported
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
 metadata JSONB
);

-- Captions table
CREATE TABLE captions (
 id TEXT PRIMARY KEY,
 clip_id TEXT REFERENCES clips(id),
 text TEXT NOT NULL,
 start_time REAL,
 end_time REAL,
 style JSONB,
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Export queue table
CREATE TABLE export_queue (
 id TEXT PRIMARY KEY,
 clip_id TEXT REFERENCES clips(id),
 platform TEXT NOT NULL, -- tiktok, youtube, instagram
 scheduled_at DATETIME,
 status TEXT DEFAULT 'queued', -- queued, exporting, completed, failed
 error_message TEXT,
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Settings table
CREATE TABLE settings (
```

```
key TEXT PRIMARY KEY,
value TEXT NOT NULL,
updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
\\
```

### ### Clip Metadata JSON

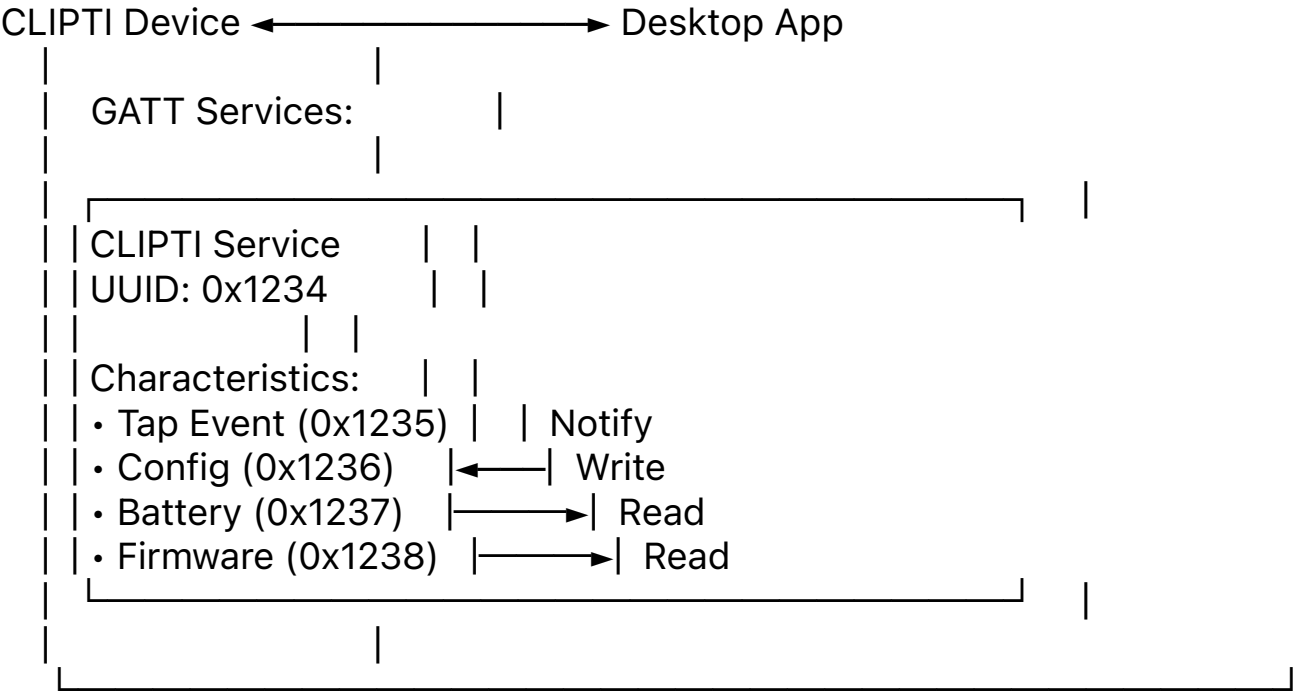
```
```json  
{  
  "id": "clip_uuid",  
  "device_id": "clipti_001",  
  "source": {  
    "camera": "gopro_hero11",  
    "resolution": "4K",  
    "fps": 30,  
    "codec": "h264"  
  },  
  "timing": {  
    "tap_timestamp": "2026-07-17T10:30:00Z",  
    "buffer_start": "2026-07-17T10:29:30Z",  
    "buffer_end": "2026-07-17T10:30:30Z"  
  },  
  "ai": {  
    "transcription": {  
      "text": "When your teammate ints but you still clutch",  
      "language": "en",  
      "confidence": 0.92  
    },  
    "captions": [  
      {  
        "text": "When your teammate ints",  
        "start": 0.0,  
        "end": 1.5  
      }  
    ],  
    "hook": {  
      "generated": "This is why you never trust your duo",  
      "hashtags": ["#gaming", "#clutch", "#viral"]  
    }  
  },  
  "status": "ready"  
}
```


}
...

Communication Protocols

BLE Communication

...



...

Tap Event Payload

```
```rust
// BLE notification when NFC tap detected
struct TapEvent {
 timestamp: u64, // Unix timestamp
 tap_count: u8, // Number of taps
 battery_level: u8, // 0-100%
 firmware_version: [u8; 3],
}
```
```

Camera Integration

GoPro Media Protocol

```
```python
GoPro WiFi Control (Hero 9+)
import requests
from urllib.parse import urlencode

class GoProCamera:
 BASE_URL = "http://10.5.5.9"

 def __init__(self):
 self.session = requests.Session()

 def start_recording(self):
 """Start GoPro recording"""
 endpoint = f"{self.BASE_URL}/gp/gpControl/execute/shutter"
 params = {"p": "shutter", "action": "start"}
 self.session.get(endpoint, params=params)

 def stop_recording(self):
 """Stop GoPro recording"""
 endpoint = f"{self.BASE_URL}/gp/gpControl/execute/shutter"
 params = {"p": "shutter", "action": "stop"}
 self.session.get(endpoint, params=params)

 def get_status(self):
 """Get current camera status"""
 endpoint = f"{self.BASE_URL}/gp/gpControl/status"
 response = self.session.get(endpoint)
 return response.json()

 def get_media_list(self):
 """List available media"""
 endpoint = f"{self.BASE_URL}/gp/gpMediaList"
 response = self.session.get(endpoint)
 return response.json()
...
```
```

RTMP Stream Capture

```

```python
Capture RTMP stream from GoPro
import ffmpeg

def capture_rtmp_stream(url, output_file, duration):
 """Capture RTMP stream to file"""
 stream = ffmpeg.input(url, t=duration)
 stream = ffmpeg.output(stream, output_file, vcodec='copy', acodec='copy')
 ffmpeg.run(stream, overwrite_output=True)
```

```

Tauri Commands

```

```rust
// src-tauri/src/commands.rs

use crate::AppState;
use tauri::State;

#[tauri::command]
pub async fn get_clips(state: State<'_, AppState>) -> Result<Vec<Clip>, String> {
 state.storage.get_all_clips().await
}

#[tauri::command]
pub async fn process_clip(
 state: State<'_, AppState>,
 clip_id: String,
) -> Result<Clip, String> {
 state.clip_processor.process(clip_id).await
}

#[tauri::command]
pub async fn export_clip(
 state: State<'_, AppState>,
 clip_id: String,
 platform: String,
) -> Result<String, String> {
 state.export_manager.queue_export(clip_id, platform).await
}
```

```

```
#[tauri::command]
pub async fn connect_device(
    state: State<'_, AppState>,
    device_id: String,
) -> Result<bool, String> {
    state.device_manager.connect(&device_id).await
}
```

```
#[tauri::command]
pub async fn get_device_status(
    state: State<'_, AppState>,
) -> Result<DeviceStatus, String> {
    state.device_manager.get_status().await
}
...
```

AI Processing

Whisper Integration

```
```rust
// src-tauri/src/ai_engine.rs

use whisper_rs::{WhisperContext, FullParams};

pub struct AIEngine {
 context: WhisperContext,
}

impl AIEngine {
 pub fn new(model_path: &str) -> Result<Self> {
 let context = WhisperContext::new(model_path)?;
 Ok(Self { context })
 }

 pub fn transcribe(&self, audio_path: &str) -> Result<Transcription> {
 let mut params = FullParams::new();
 params.set_language(Some("en"));
 }
}
```

```

let mut state = self.context.new_state()?;
state.full(params, audio_path)?;

// Extract transcription
let text = state.get_text_chunks()
 .map(|chunk| chunk.text.to_string())
 .collect::<Vec<_>>()
 .join(" ");

Ok(Transcription { text, language: "en".into() })
}
}
...

```

### ### Hook Generation (DeepSeek)

```

```rust
// src-tauri/src/ai_engine.rs

pub async fn generate_hook(&self, transcription: &str) -> Result<Hook> {
    let prompt = format!(
        "Generate a viral TikTok hook and hashtags for this clip:\n\n{}\n\n\
        Output JSON: {{\n\"hook\": \"...\",\n\"hashtags\": [\"#tag1\", \"#tag2\"]\n}}",
        transcription
    );

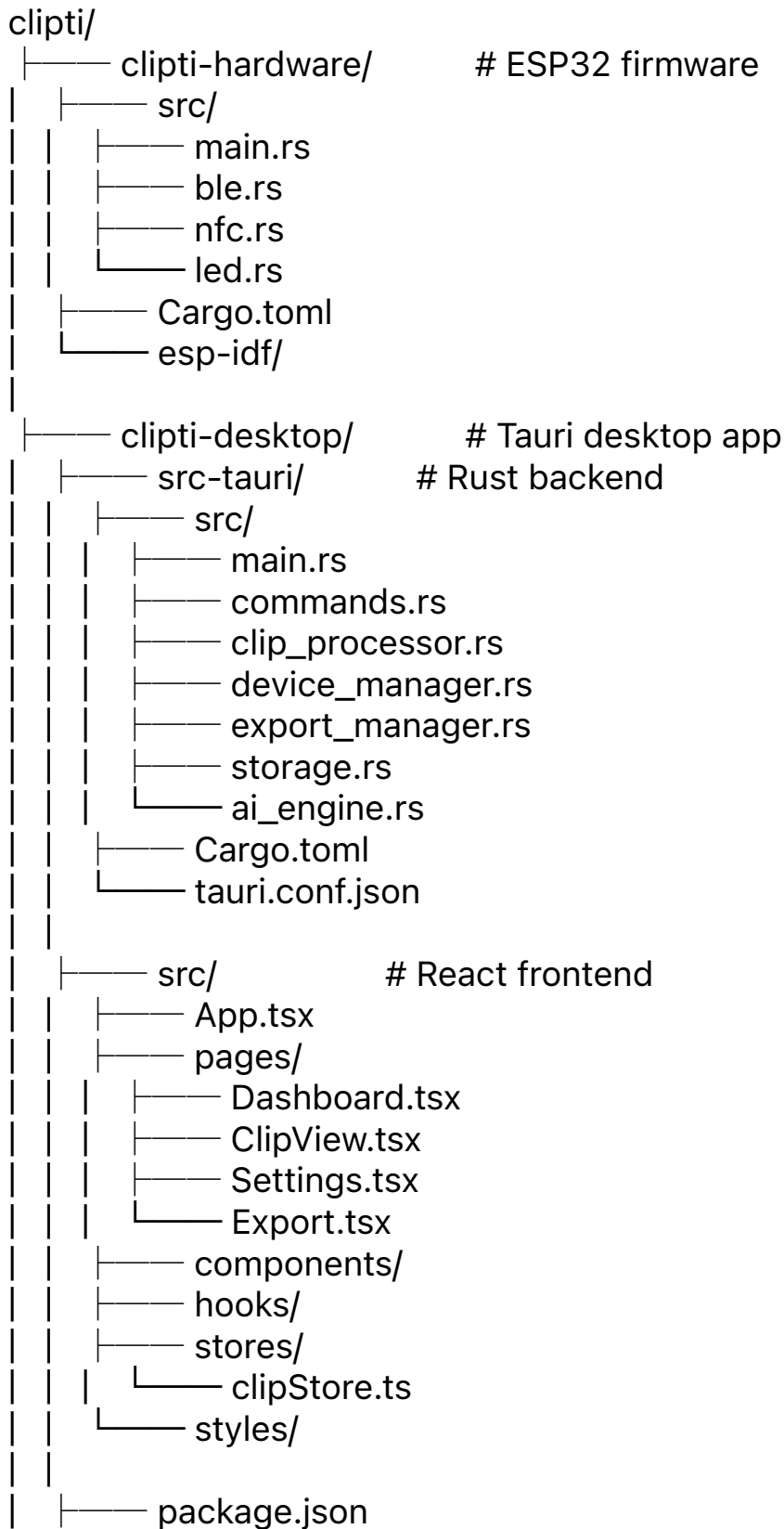
    let response = reqwest::Client::new()
        .post("https://api.deepseek.com/v1/chat/completions")
        .header("Authorization", format!("Bearer {}", self.api_key))
        .json(&json!({
            "model": "deepseek-chat",
            "messages": [{ "role": "user", "content": prompt }],
            "temperature": 0.8
        }))
        .send()
        .await?;

    let hook: Hook = response.json().await?;
    Ok(hook)
}
...

```

File Structure

...



```

|   └── vite.config.ts
|
|   └── docs/
|       ├── ARCHITECTURE.md
|       ├── API.md
|       └── HARDWARE.md
|
|   └── README.md
|
...

```

Dependencies

Rust Dependencies (Cargo.toml)

```

```toml
[dependencies]
tauri = { version = "2.0", features = ["shell-open"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
tokio = { version = "1.0", features = ["full"] }
rusqlite = { version = "0.31", features = ["bundled"] }
ffmpeg-next = "7.0"
reqwest = { version = "0.12", features = ["json"] }
whisper_rs = "0.4"
uuid = { version = "1.0", features = ["v4"] }
chrono = { version = "0.4", features = ["serde"] }
tracing = "0.1"
thiserror = "1.0"
anyhow = "1.0"
```

```

Frontend Dependencies (package.json)

```

```json
{
 "dependencies": {
 "react": "^18.3.0",
 "react-dom": "^18.3.0",
 "@tauri-apps/api": "^2.0",
 "zustand": "^4.5.0",

```

```

 "react-router-dom": "^6.23.0",
 "lucide-react": "^0.400.0",
 "date-fns": "^3.6.0"
 },
 "devDependencies": {
 "@types/react": "^18.3.0",
 "typescript": "^5.4.0",
 "vite": "^5.4.0",
 "@vitejs/plugin-react": "^4.3.0",
 "tailwindcss": "^3.4.0",
 "autoprefixer": "^10.4.0",
 "postcss": "^8.4.0"
 }
}
` ``

```

---

## ## Configuration

### ### Tauri Configuration (tauri.conf.json)

```

`` `json
{
 "productName": "CLIPTI",
 "identifier": "com.clipti.app",
 "version": "1.0.0",
 "build": {
 "devtools": true
 },
 "app": {
 "windows": [
 {
 "title": "CLIPTI Dashboard",
 "width": 1280,
 "height": 800,
 "minWidth": 1024,
 "minHeight": 600,
 "resizable": true,
 "fullscreen": false
 }
]
 },
}

```



```

"security": {
 "csp": "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'"
},
"bundle": {
 "active": true,
 "targets": "all",
 "icon": [
 "icons/32x32.png",
 "icons/128x128.png",
 "icons/128x128@2x.png",
 "icons/icon.icns",
 "icons/icon.ico"
]
}
}
...

```

---

## ## Testing Strategy

### ### Unit Tests

```

```rust
// src-tauri/tests/clip_processor.rs

#[cfg(test)]
mod tests {
  use crate::clip_processor::ClipProcessor;

  #[tokio::test]
  async fn test_extract_clip() {
    let processor = ClipProcessor::new();
    let clip = processor.extract("test_buffer.mp4", 30.0).await;
    assert!(clip.is_some());
  }

  #[tokio::test]
  async fn test_transcription() {
    let engine = AIEngine::new("models/whisper.bin").unwrap();
    let result = engine.transcribe("test_audio.wav").await;

```

```
        assert(!result.text.is_empty());
    }
}
```

Integration Tests

```
```typescript
// src/__tests__/integration.test.tsx

describe('CLIPTI Integration', () => {
 it('should connect to device', async () => {
 const connected = await invoke('connect_device', { deviceId: 'clipti_001' });
 expect(connected).toBe(true);
 });

 it('should process clip through pipeline', async () => {
 const clip = await invoke('process_clip', { clipId: 'test_clip' });
 expect(clip.status).toBe('ready');
 });
});
```
```

Performance Targets

| Metric | Target | Measurement |
|-------------------------|---------|------------------------|
| Clip extraction latency | < 2s | From tap to saved file |
| Transcription time | < 5s | Per 30s clip |
| Memory usage | < 500MB | Desktop app |
| Battery life | 8 hours | Hardware device |
| BLE latency | < 100ms | Tap to notification |
| App startup | < 3s | Cold start |

Security Considerations

| Concern | Mitigation |
|---------|------------|
| ----- | ----- |

| BLE interception | Encryption (BLE 5.0 Secure Connections) |
| Clip storage | Local only, optional cloud encryption |
| API keys | Environment variables, never hardcoded |
| OTA firmware | Signed updates, secure boot |
| Data privacy | No cloud by default, user consent for upload |

Last updated: July 2026